



{ }

# 31 Days of React Design Patterns

## Container-Presenter Pattern

A simple pattern that separates logic from UI.  
Learn how to build cleaner React components.

### Logic

```
const [users, setUsers] =  
  useState([]);  
  
useEffect(() => {  
  fetch('/api/users')  
    .then(res => res.json())  
    .then(data =>  
      setUsers(data));  
}, []);
```

 State API Logic

### UI

 John Doe  
john@example.comTotal Users  
**1,248**

Add User

Refresh

Recent Users



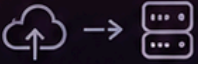
## THE PROBLEM

# Why Components Become Messy

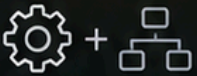
Too many responsibilities inside one component create complexity.

```
const App = () => {  
  return (  
    <div>  
      <Component />  
    </div>  
  )  
}
```

✗ API Calls



✗ Business Logic



BigComponent.jsx

```
1  import { useState, useEffect } from 'react'  
2  function BigComponent() {  
3    useEffect(() => {  
4      fetch('/api/users')  
5        .then(res => res.json())  
6        .then(data => setUsers(data))  
7    }, [])  
8  
9    const [users, setUsers] = useState([])  
10   const [loading, setLoading] = useState(true)  
11   const [error, setError] = useState(null)  
12  
13   const activeUsers = users.filter(u => u.active)  
14   const sortedUsers = activeUsers.sort(  
15     (a, b) => b.createdAt - a.createdAt  
16   )  
17   const total = activeUsers.length  
18   const canAdd = total < 10  
19  
20   return (  
21     <div className="p-4">  
22       <h2>Users</h2>  
23       <button onClick={handleAdd}  
24         className="btn btn-primary">  
25         Add User  
26       </button>  
27       <ul>  
28         {sortedUsers.map(user => (  
29           <UserCard key={user.id} user={user} />  
30         ))}  
31       </ul>  
32     </div>  
33   )  
34 }
```

```
function  
Component() {  
  // Too much  
  going on  
}
```

✗ State Management



✗ UI Rendering



RESULT

**Messy code and harder maintenance.**



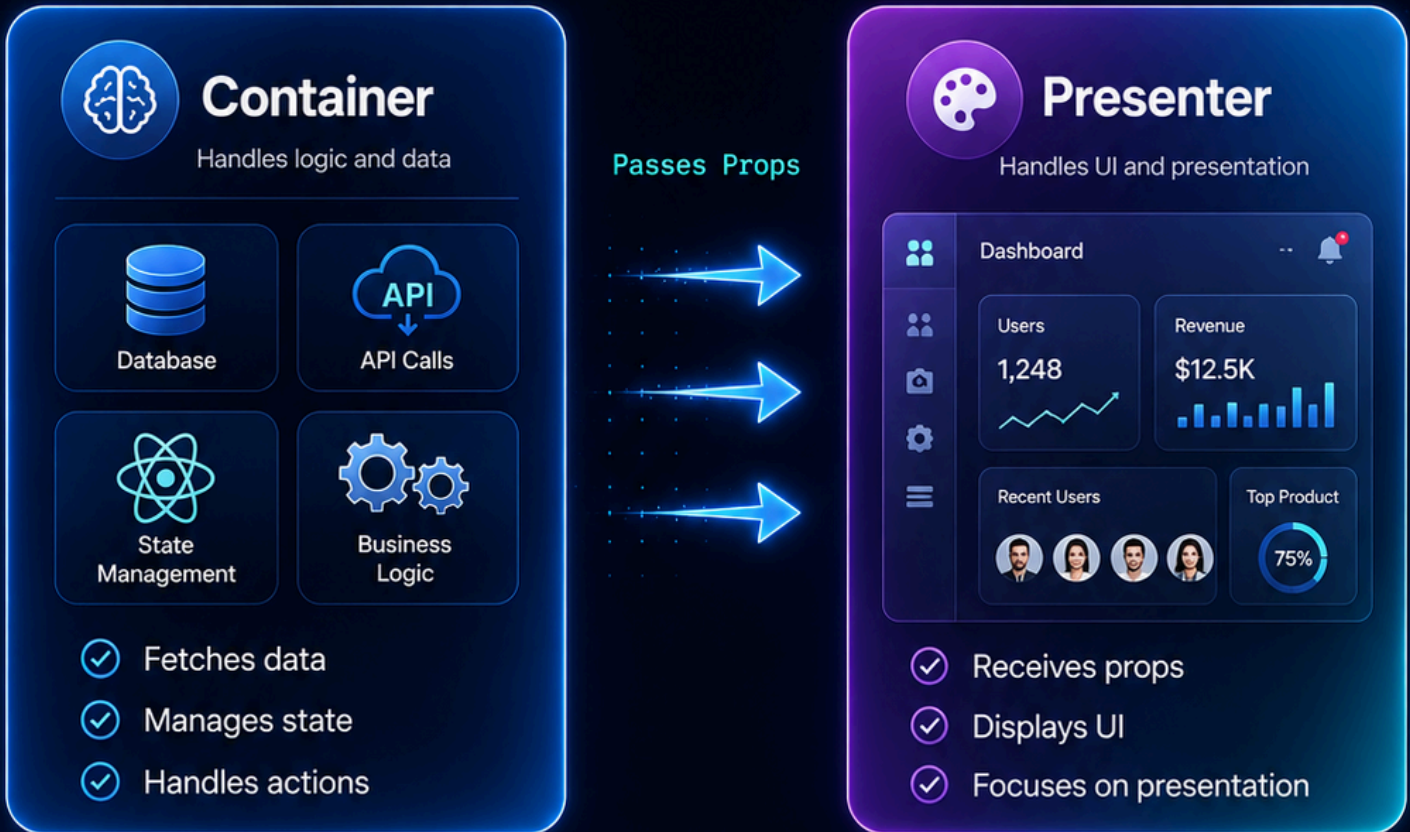
{ EVERYTHING IN ONE FILE = GROWING COMPLEXITY }



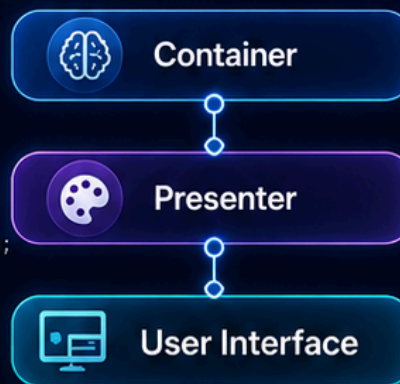


# What Is The Container-Presenter Pattern?

Separate responsibilities to create cleaner and more maintainable components.



```
function Container() {
  const [users, setUsers] =
    useState([]);
  const fetchUsers = async () => {
    const res = await fetch('/api/users');
    setUsers(await res.json());
  };
  return <Presenter users={users} />;
}
```



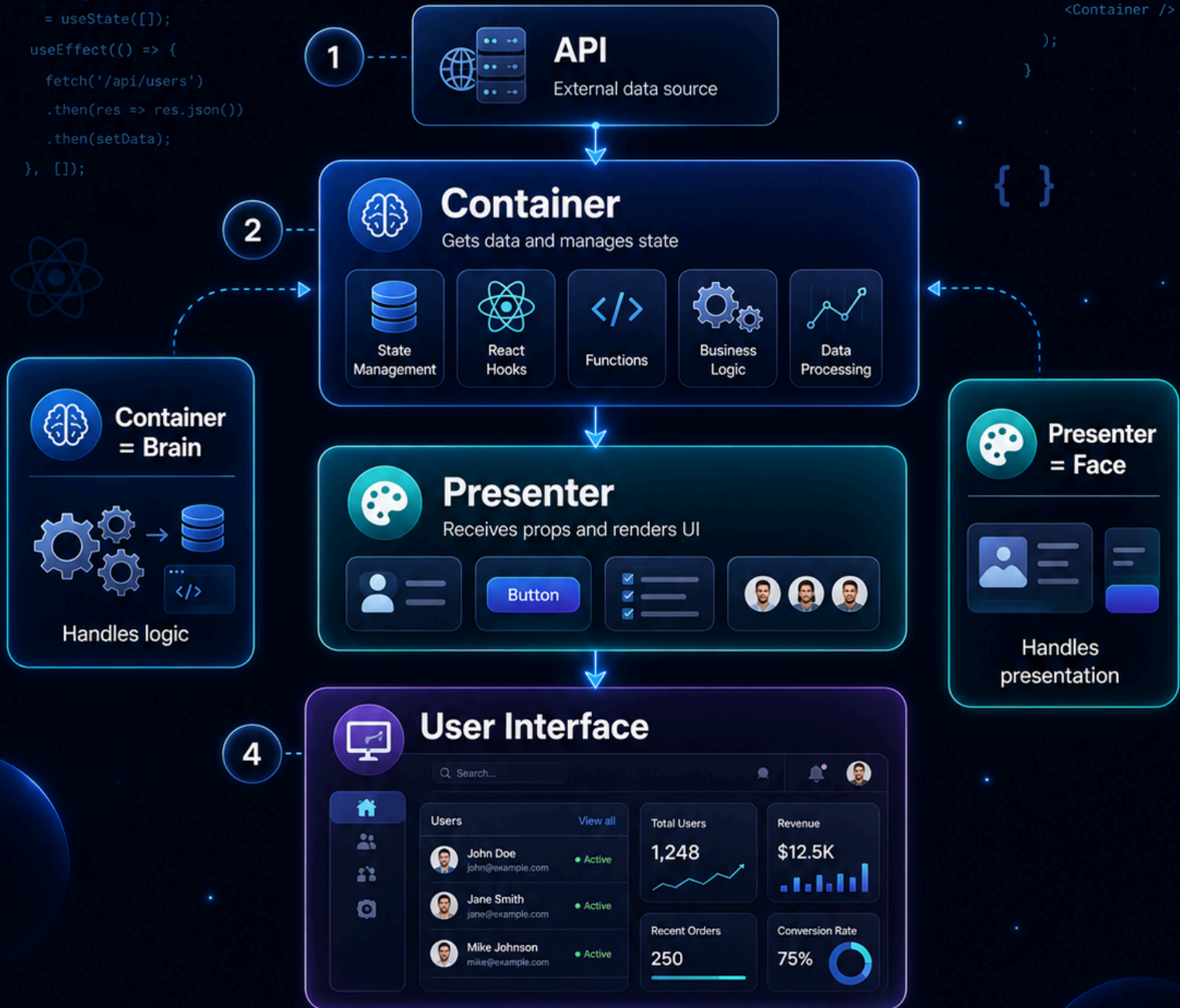
```
function Presenter({ users }) {
  return (
    <div className="user-list">
      {users.map(user => (
        <UserCard key={user.id}
          user={user} />
      ))}
    </div>
  );
}
```



Separation of responsibilities leads to cleaner code.

# How The Pattern Works

Data flows through the Container before reaching the UI.



Container = Brain



Presenter = Face





# Container & Presenter in Code

Separate logic from presentation for cleaner React components.

```
const App = () => {
  return (
    <Container />
  );
};
```

```
function UserList({ users }) {
  return (
    <ul>...</ul>
  );
}
```



## Container

```
Container.jsx x JSX
1 import { useState, useEffect } from 'react';
2
3 export default function Container() {
4   const [users, setUsers] = useState([]);
5
6   useEffect(() => {
7     fetch('/api/users')
8       .then(res => res.json())
9       .then(data => setUsers(data));
10  }, []);
11
12  return <UserList users={users} />;
13 }
```



State Management



Data Fetching



Business Logic

Passes Props



## Presenter

```
UserList.jsx x JSX
1 export default function UserList({ users }) {
2   return (
3     <div className="user-list">
4       {users.map(user => (
5         <div className="user-card" key={user.id}>
6           <img src={user.avatar} alt={user.name} />
7           <div className="info">
8             <h4>{user.name}</h4>
9             <p>{user.email}</p>
10          </div>
11        </div>
12      ))}
13    </div>
14  );
15 }
```



UI Rendering



Displays Data



Presentation Layer



## Container

Handles:

- ✓ Logic
- ✓ State
- ✓ Data

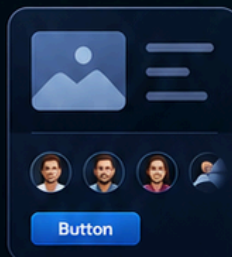
VS



## Presenter

Handles:

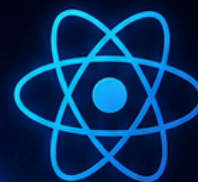
- ✓ UI
- ✓ Display
- ✓ Rendering



Container handles **logic**.



Presenter handles **display**.



# Why Developers Love This Pattern

Cleaner architecture leads to easier maintenance and better collaboration.

```
const Component = () => {  
  return (  
    <div className="app">  
      /* clean architecture */  
    </div>  
  );  
};
```



## Cleaner Components

Reduce complexity and improve readability.



## Easier Testing

Logic and UI become simpler to test independently.



## Better Reusability

Reuse presentational components across multiple screens.



## Easier Collaboration

Frontend teams can work more efficiently.

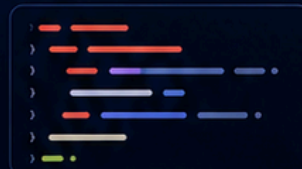


## More Maintainable Code

Future updates become easier and less painful.



## Messy Code



## Cleaner Structure



## Scalable Application



Clean code today  
saves time tomorrow.





WHEN TO USE IT



{ }

# When Should You Use This Pattern?

</>

```
const Component = () => {  
  return (  
    <div className="app">  
      {/* clean architecture */}  
    </div>  
  );  
};
```

Not every component needs a pattern  
but some definitely benefit from it.

## Container- Presenter Pattern

### Ideal For

-  Complex components
-  Reusable interfaces
-  Scalable applications
-  Collaborative projects

### ✓ Lots of Logic



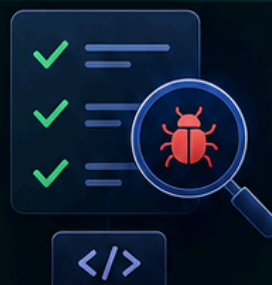
Components handling complex logic become easier to manage.

### ✓ Shared UI



Reuse the same presentation layer across different views.

### ✓ Easier Testing



Separate logic and UI to simplify testing.

### ✓ Team Projects



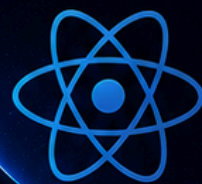
Clear separation helps teams work more efficiently.



**More complexity**  
= **More value** from separation







# Not Every Component Needs A Pattern

Avoid unnecessary complexity when simplicity already works.



## Simple Component

UserCard.jsx

```
1 export default function
2   UserCard({ user }) {
3     return (
4       <div className="card">
5         <img src={user.avatar}
6           alt={user.name} />
7         <h4>{user.name}</h4>
8         <p>{user.email}</p>
9       </div>
10    );
11  }
```

src

components

UserCard.jsx



John Doe

john@example.com

View Profile

VS



## Over-Engineered Structure

src

containers

UserCardContainer.jsx

presenters

UserCardPresenter.jsx

hooks

useUserCard.js

services

userService.js

types

user.types.js

Container  
Logic & State



Hook  
Custom Logic



Service  
API Calls



Presenter  
UI Render



- Extra files
- More maintenance
- Harder navigation



## Component Is Very Small

```
const Button = () => {
  <button>Click me</button>
};
```



Simple components rarely need additional layers.



## Logic Is Minimal

```
const sum = (a, b) => {
  return a + b;
};
```



Adding containers may create more complexity than value.



## Too Many Files



Over-engineering hurts maintainability.



**Patterns solve problems.**  
**They shouldn't create new ones.**



Keep it simple until complexity demands separation.





# Container-Presenter In A Real App

An e-commerce product page is a perfect example of separating logic from UI.

```
const ProductPage = () => {
  return (
    <Container>
      <Presenter />
    </Container>
  );
};
```

1



## Products API

GET /api/products



2



## Products Container

### Handles

- ✓ Fetches products
- ✓ Loading state
- ✓ Product filtering



3



## Product Grid Presenter

### Displays

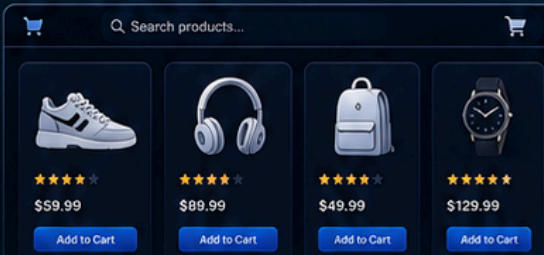
- ✓ Product cards
- ✓ Prices
- ✓ Ratings



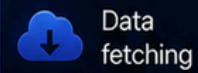
4



## User Interface



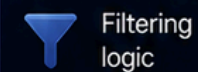
## Container



Data  
fetching



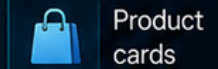
Loading  
states



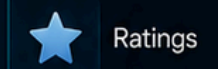
Filtering  
logic

Business logic  
lives here.

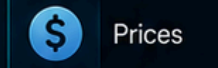
## Presenter



Product  
cards



Ratings



Prices

UI rendering  
lives here.

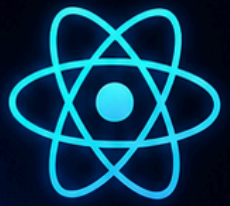
```
function useProducts() {
  const [products, setProducts] =
    useState([]);
  const [loading, setLoading] =
    useState(true);
  ...
}
```



Clean separation of concerns  
creates **scalable** applications.

Logic belongs in containers. UI belongs in presenters.





# Keep Components Focused

The **Container-Presenter** Pattern helps keep React components focused on one responsibility.



Logic belongs in containers



Data Fetching



State Management



Business Logic

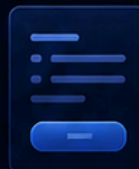


UI belongs in presenters



\$59.99

★ ★ ★ ★ ★



UI Components



Interactive Elements



Presentation Layer



Have you ever worked on a React component that tried to **do everything**?

Share your experience in the comments.

```
const App = () => {  
  return (  
    <div>  
      <Component />  
    </div>  
  );  
};
```

```
const  
state = {  
  loading: true,  
  data: [],  
  error: null  
};
```



mtayyabrawan



Follow for more React patterns &  
frontend architecture insights

Frontend Developer • React Enthusiast • Building in Public